

Compliance Reviewer — Demo Report

Tool: EPIC Compliance Reviewer (prototype)

Steering doc / spec: T&S R&C Unified Controls Framework-APP.md (PG&E T&S R&C Unified Controls Framework, app-applicable subset — 76 NIST 800-53 controls)

Target repo: ai-code-reviewer/ (Application-Codebase/ — Node.js/TypeScript Fastify API + CLI scanner)

Scan date: 2026-07-07

Rule set: 18 rules derived from the framework (6 hard-criterion + 12 presence) — the enforceable subset of the 76 controls

What this demo shows. Using **only** the -APP framework file as the specification, this is exactly what an EPIC compliance stage would emit for one real repo: each finding cites a NIST control ID, states the file's requirement, and gives a verdict backed by code evidence (file:line). It also shows — honestly — how many controls in the framework simply **cannot** be judged from source code and are reported as MANUAL or N/A rather than falsely passed.

Executive summary

Verdict	Count	Meaning
✅ PASS	2	Control satisfied, code evidence present
⚠️ PARTIAL	5	Mechanism present but incomplete
❌ FAIL	3	Control applies; code shows it is not met
— N/A	6	Precondition absent in this app (e.g. no login → no logout)
📄 MANUAL	2	Real control, needs human/runtime attestation — not repo-checkable

Gate result (HARD rules only): PASS-WITH-WARNINGS. No HARD-rule FAIL that isn't N/A. The target has no interactive user-auth surface, so the strongest access-control rules don't apply; the genuine gaps (unauthenticated API, plain HTTP) fall on presence rules, reported as FAIL/warning.

Context on the target: ai-code-reviewer is a scanner service — a Fastify REST API + CLI, no end-user login, accounts, or sessions. Its only authentication is machine-to-machine GitHub **webhook HMAC**. That shape is *why* so many access-control controls read N/A — and it's a faithful demonstration that the tool distinguishes "not applicable" from "failed."

HARD rules (concrete criterion in the framework)

✗ — AC-06-00 — Account lockout — N/A

Requirement (file): "Enforce a limit of no more than **10** consecutive invalid logon attempts by a user during any **60-minute** period... lock the account or node for at least **15 minutes**."

Evidence: No login/authentication path exists in the codebase. No password, session, or credential-verification logic (grep for login|password|loginAttempts|session across src/ returns only LLM-prompt text in platform/*-context.ts, not implementation).

Verdict: N/A — no authentication surface to apply a lockout to. *Would become gate-relevant the moment a login is added.*

— AC-07-00 — System-use notification banner — N/A

Requirement (file): Display a use-notification banner with four mandated statements (PG&E system; usage monitored; unauthorized use prohibited; use implies consent) before granting access. **Evidence:** No interactive logon interface. API endpoints are programmatic (REST/JSON); the framework itself states banners apply "only for access via logon interfaces with human users." **Verdict:** N/A.

— AC-08-00 — Concurrent session limit (2) — N/A

Requirement (file): "Limit the number of concurrent sessions for all individual and shared accounts to **two**." **Evidence:** No session management exists (no express-session, no session store, no cookie-based sessions). Stateless API keyed by request. **Verdict:** N/A.

⚠ AU-02-00 — Audit records contain 6 fields — PARTIAL

Requirement (file): Audit records must establish: (a) what type of event, (b) when, (c) where, (d) source, (e) outcome, (f) identity of any individual. **Evidence:**

- API path uses Fastify's built-in pino logger — `Fastify({ logger: true })` at Application-Codebase/src/api/server.ts:21 — which emits **time + level + message** and per-request method/URL/status. That covers *when*, partial *what*, partial *outcome*.
- CLI path uses bare `console.log/console.error` (69 calls across src/; e.g. Application-Codebase/src/cli.ts:129) — **no structured fields**.
- No consistent capture of *source*, *outcome*, or *identity* (there is no user identity in the system). **Verdict:** PARTIAL — request logging covers ~2–3 of 6 fields; CLI logging covers none. Recommendation: adopt one structured logger emitting all six fields.

⚠ AU-06-00 — UTC time stamps from internal clock — PARTIAL

Requirement (file): "Use internal system clocks to generate time stamps for audit records" synchronized to UTC. **Evidence:** Health endpoint uses `new Date().toISOString()` (UTC) at Application-Codebase/src/api/server.ts:61. Pino default timestamps are epoch-

millis from the system clock (UTC-based) but not ISO-8601 formatted. CLI logs carry no timestamp at all. **Verdict:** PARTIAL — timestamps are UTC-sourced where present, but inconsistent and absent from CLI output.

⚠️ AC-12-00 — Log the enumerated security event types — **PARTIAL**

Requirement (file): Log security-relevant event types — authentication events, authorization grants/denials, config changes, intrusion/anomaly events, privilege escalation, etc. **Evidence:** The app logs **operational** events (scan progress, HTTP requests) but no **security** events — there are no auth/authz events to log (no auth), and no logging of config changes or the webhook-validation outcome. Webhook HMAC failures are rejected at Application-Codebase/src/github/webhook-validator.ts:15 but the rejection is not audit-logged as a security event. **Verdict:** PARTIAL — a security-relevant event (failed HMAC) exists and is unlogged; most other event types are N/A for this app.

PRESENCE rules (mechanism presence — framework states no threshold)

❌ IA-02-00 — Authenticate users — **FAIL**

Requirement (file): Uniquely identify and authenticate users (or processes acting on behalf of users). **Evidence:** REST endpoints (POST /repos, POST /repos/:id/scan, GET /repos/:id/scans/...) have **no authentication middleware** — Application-Codebase/src/api/server.ts:25-58 registers only a Content-Type guard and routes; no preHandler auth, no API-key check, no bearer validation. Only the /webhooks/github route authenticates (HMAC). **Verdict:** FAIL — the control-plane API is unauthenticated. (Warning-level; not a HARD gate rule.)

❌ AC-03-00 — Server-side access enforcement — **FAIL**

Requirement (file): Enforce approved authorizations for logical access; deny by default; enforce server-side. **Evidence:** No authorization layer on the API. Any caller who can reach 0.0.0.0:3000 can register repos and trigger scans. **Verdict:** FAIL — no access enforcement on non-webhook routes.

❌ SC-08-00 — Encryption in transit — **FAIL**

Requirement (file): Protect the confidentiality/integrity of transmitted information (TLS). **Evidence:** Server listens on plain HTTP — app.listen({ port, host: '0.0.0.0' }) at Application-Codebase/src/api/server.ts:71. No TLS termination in-app (grep for https/tls/cert finds only outbound URLs and git-clone token injection). **Verdict:** FAIL in isolation. *Note:* in a real deployment TLS is typically terminated at an ALB/ingress (IaC), which this repo doesn't contain — see MANUAL note. Flagged so the reviewer confirms edge TLS exists.

⚠️ SI-07-00 — Software/artifact integrity — **PARTIAL**

Evidence: Genuine integrity control present: GitHub webhook HMAC-SHA256 with constant-time compare — Application-Codebase/src/github/webhook-validator.ts:15-19 uses createHmac + timingSafeEqual. No dependency SBOM, no lockfile-integrity gate, no artifact signing. **Verdict:** PARTIAL.

⚠️ AU-11-00 — Audit record generation — **PARTIAL**

Evidence: Fastify request logging is on (logger: true). No retention policy, no tamper-evident store. **Verdict:** PARTIAL.

⚠️ CM-06-00 — Least functionality — **PARTIAL**

Evidence: Single service on port 3000, bound to 0.0.0.0 (all interfaces) — broader than needed. No documented port/service minimization. **Verdict:** PARTIAL.

— AC-02-04 — Audit account-management actions — **N/A**

Evidence: No user accounts to create/modify/disable. N/A.

— AU-08-00 — Non-repudiation time stamps — **N/A / PARTIAL**

Evidence: No user actions to attribute (no identity). Timestamps exist for operational logs. N/A for the non-repudiation intent.

— CP-10-00 — Unique user authentication — **N/A**

Evidence: No users. N/A.

— CM-05-00 — Secure baseline configuration — **N/A**

Evidence: infra/ is an empty CDK scaffold (cdk.json + package.json only; no stack code). No config to evaluate. N/A until IaC exists.

📄 SC-28-00 — Encryption at rest — **MANUAL**

Evidence: Persistence is a local SQLite file (sql.js). At-rest encryption is a storage/infra property (EBS/KMS), not visible in this repo (no IaC). Requires deployment-layer attestation. **Verdict:** MANUAL.

📄 AU-10-00 — Audit retention — **MANUAL**

Evidence: Retention is a log-pipeline/infra property. Not determinable from source. **MANUAL.**

What this demonstrates

1. **The tool produces auditor-grade, control-keyed findings** — every line cites a NIST ID, the framework's own requirement, and code evidence with `file:line`. That's the differentiator vs. Wiz/SonarQube.
2. **It is honest about coverage.** Of the 76 app-applicable controls, only ~6 have a hard pass/fail rule in this file and ~12 more are presence-checkable. The tool marks the rest MANUAL/N/A — it does **not** fake-pass process controls.
3. **It distinguishes N/A from FAIL.** The target has no login, so lockout/banner/session controls are correctly N/A, while real gaps (unauthenticated API, plain HTTP) surface as FAIL.
4. **The ceiling of "this file alone" is low.** To convert the 12 presence checks into real gates (thresholds, exact header values, JWT claim validation), we need security-baseline.md + the TS-Controls-AIDLC-Mapping.md crosswalk. **Recommend approving Phase 2** to make the gate meaningfully enforce more than 6 controls.

Caveats

- The -APP.md source is a **degraded PDF extraction** — control *names* are shifted and requirement prose is run-on. Rule text above was reconstructed from the prose block inside each control. A clean source doc is needed before hard-coding thresholds.
- This prototype run was performed as a **manual analyst pass** to illustrate output shape. The production tool automates this via a deterministic + LLM engine in an EPIC stage (see ../compliance-reviewer.md).
- Verdicts reflect the **repo only**. Deployment-layer controls (edge TLS, at-rest encryption, log retention) may be satisfied by infrastructure this repo doesn't contain — hence MANUAL, not FAIL, where appropriate.